

SMART TROLLEY SYSTEM USING ESP32 AND RFID

A Course Project Report

Embedded Systems

Project Team

G. Neha Sree

SR UNIVERSITY

Academic Year 2025–2026

ABSTRACT

The Smart Shopping Trolley is an embedded system designed to simplify the shopping experience by reducing billing time and helping users manage their expenses efficiently. This system is built using an ESP32 microcontroller integrated with an RFID reader, keypad, LCD display, buzzer, servo motor, and Bluetooth module. The user begins by entering a budget through the keypad, which is displayed on the LCD screen. As items are added to the trolley, the RFID reader scans each product's unique tag, retrieves its price from a predefined database, and updates the total amount in real time.

The system continuously compares the total cost with the user-defined budget. If the budget limit is exceeded, an alert is generated using a buzzer and a warning message is displayed on the LCD. The user is given the option to either continue shopping by overriding the limit or cancel the item. Additionally, billing details are transmitted via Bluetooth for easy tracking. A servo mechanism is used to simulate smart access to the trolley. This system enhances shopping convenience, minimizes manual billing errors, and provides a cost-effective solution for automated retail environments.

1. INTRODUCTION

In today's fast-paced world, shopping in supermarkets and malls often involves long queues at billing counters and difficulty in tracking expenses while purchasing items. Customers usually add products to their cart without knowing the exact total cost, which may lead to overspending. To overcome these issues, the Smart Shopping Trolley system is developed using embedded system technology.

This project aims to provide a smart and efficient shopping experience by integrating an ESP32 microcontroller with RFID technology. Each product is attached with an RFID tag, which contains a unique identification number. When the product is scanned using the RFID reader, its details such as name and price are retrieved and displayed on an LCD screen. The system also allows users to set a budget using a keypad, helping them monitor their spending in real time. If the total amount exceeds the set budget, the system alerts the user through a buzzer and display message. Bluetooth communication is used to send billing information wirelessly.

2. SYSTEM DESCRIPTION

2.1 Block Diagram

SMART TROLLEY SYSTEM USING ESP32 AND RFID

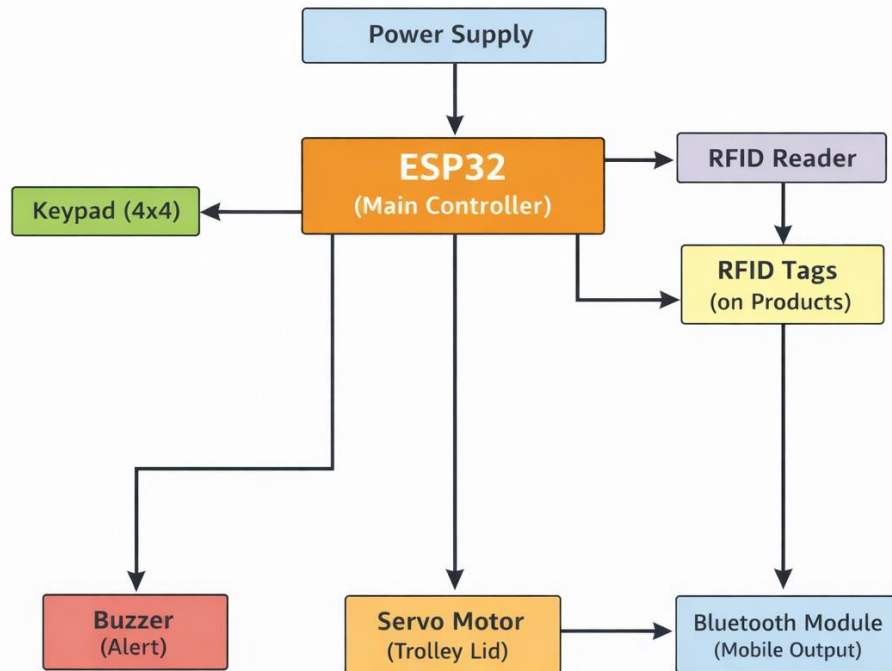


Fig. 2.1: Block Diagram of Smart Shopping Trolley System

The Smart Shopping Trolley system is designed to automate the billing process and help users manage their shopping budget efficiently. The system is built around the ESP32 microcontroller, which acts as the central processing unit and controls all connected components.

Initially, the user sets a budget using a 4x4 keypad. The entered value is displayed on a 16x2 LCD screen. As the user adds items to the trolley, each product is scanned using the RFID reader (MFRC522). Every product has a unique RFID tag, and its corresponding name and price are stored in the system database. When a tag is scanned, the system matches the UID with stored data, retrieves the product details, and updates the total bill.

The updated total and product name are displayed on the LCD, and the information is also sent via Bluetooth to a mobile device. A buzzer is used to provide alerts. If the total cost exceeds the predefined budget, the system generates a warning and allows the user to either continue shopping by overriding the limit or cancel the item. Additionally, a servo motor is used to simulate a smart lid mechanism.

2.2 Working Principle

Each product has a unique RFID tag. When scanned, the RFID reader sends the tag's UID to the ESP32, which matches it with stored data to get the product name and price. The LCD displays the details and updates the total bill. If the total exceeds the set budget, a buzzer alert is given. The user can continue or cancel, and the bill is sent via Bluetooth.

2.3 RFID SCANNING PROCESS

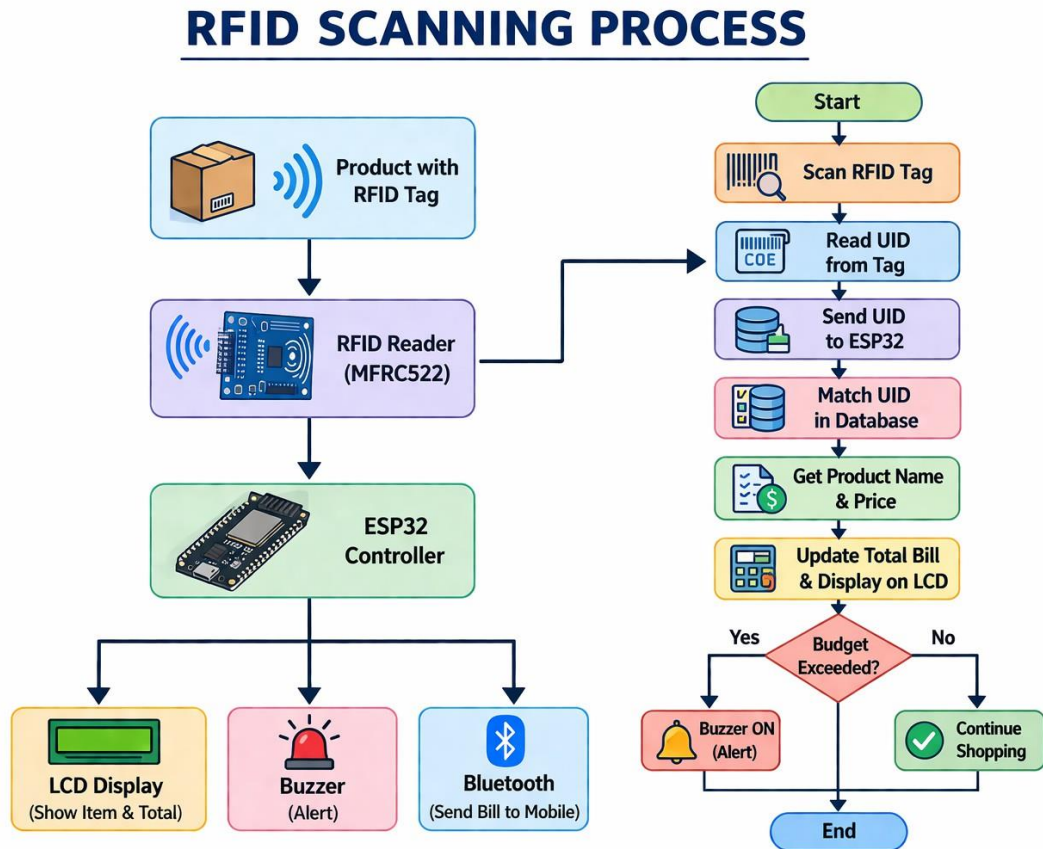


Fig. 2.2: RFID Scanning Process

3. HARDWARE & SOFTWARE TOOLS

3.1 Hardware Components

Component	Function
ESP32 Microcontroller	Handles processing and communication (Bluetooth).
MFRC522 RFID Reader	Facilitates contactless scanning.
RFID Tags (Passive)	Attached to products for unique identification.
LCD Display (16x2 with I2C)	Provides the user interface.
Power Supply	Provides stable 3.3V and 5V power.
Keypad (4x4)	Used for manual input of the shopping budget.
Servo Motor	Acts as a locking mechanism / trolley lid actuator.
Bluetooth / Mobile Integration	Provides a secondary interface to view the final bill.
Buzzer	Provides audio alerts when budget exceeds.

3.2 Pin Configuration

Component	Pin	ESP32 Pin	Purpose
RFID (MFRC522)	VCC	3.3V	Power Supply
	GND	GND	Ground
	RST	GPIO 22	Reset
	SDA (SS)	GPIO 5	SPI Chip Select
	MOSI	GPIO 23	SPI Data Out
	MISO	GPIO 19	SPI Data In
	SCK	GPIO 18	SPI Clock
LCD (16x2 I2C)	VCC	5V	Power Supply
	GND	GND	Ground
	SDA	GPIO 21	I2C Data Line
	SCL	GPIO 17	I2C Clock Line
Servo Motor	Signal	GPIO 4	Control Signal
	VCC	5V	Power Supply
	GND	GND	Ground
Buzzer	+	GPIO 15	Alert Signal
	-	GND	Ground
Keypad (4x4)	Row Pins	GPIO 32, 33, 25, 26	Row Inputs
	Column Pins	GPIO 27, 14, 12, 13	Column Inputs

4. IMPLEMENTATION

The Smart Shopping Trolley system is implemented using ESP32 by integrating RFID, keypad, LCD, buzzer, servo motor, and Bluetooth modules. Initially, when the system is powered on, the user is prompted to enter a budget using the keypad. The entered value is displayed on the LCD and stored in the system. After setting the budget, the system continuously waits for RFID tags to be scanned.

When a product is placed near the RFID reader, its UID is detected and matched with the predefined product database. If a match is found, the product name and price are retrieved, and the total bill is updated. The details are displayed on the LCD and also sent via Bluetooth. If the total amount exceeds the set budget, the system generates an alert using a buzzer and displays a warning message. The user can choose to continue shopping by overriding the limit or cancel the item.

Additionally, a servo motor is used to open the trolley lid after a successful scan. The final bill is generated when the user presses the stop button, and the system displays the total amount.

4.1 Circuit Diagram

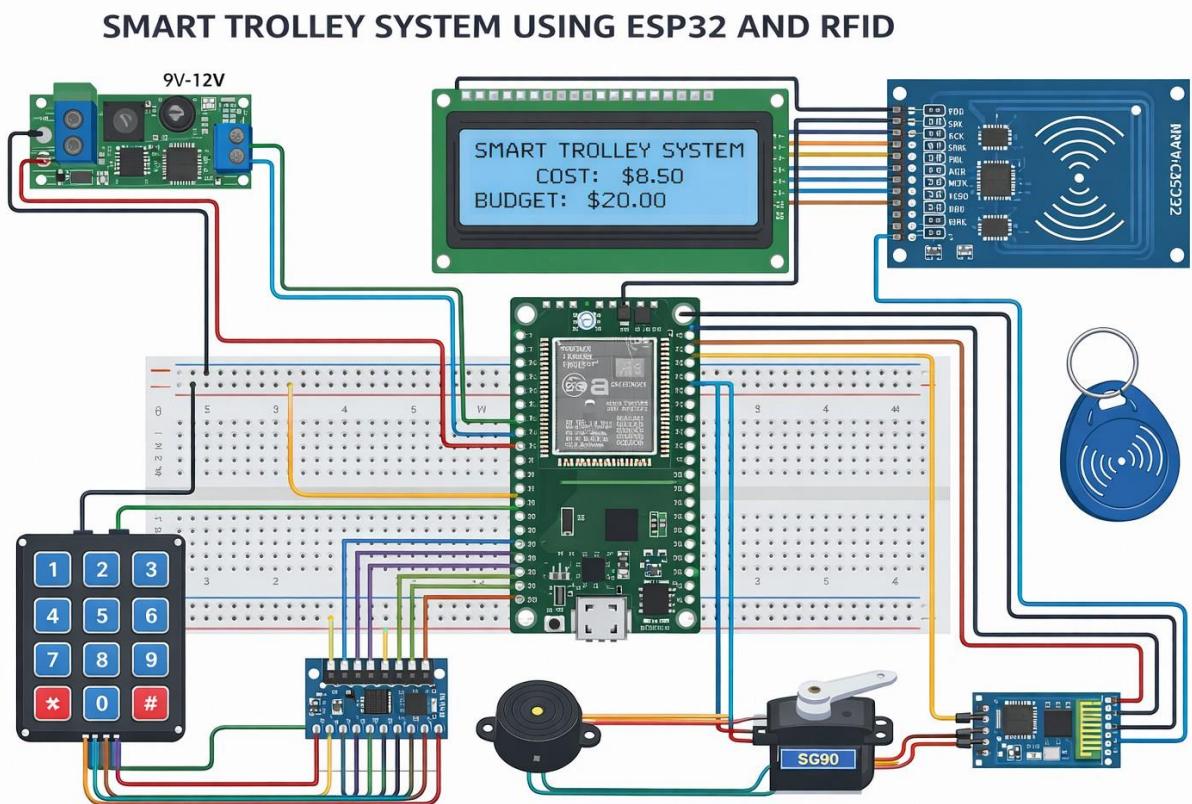


Fig. 4.1: Circuit Diagram of the Smart Trolley System

4.2 Hardware Setup



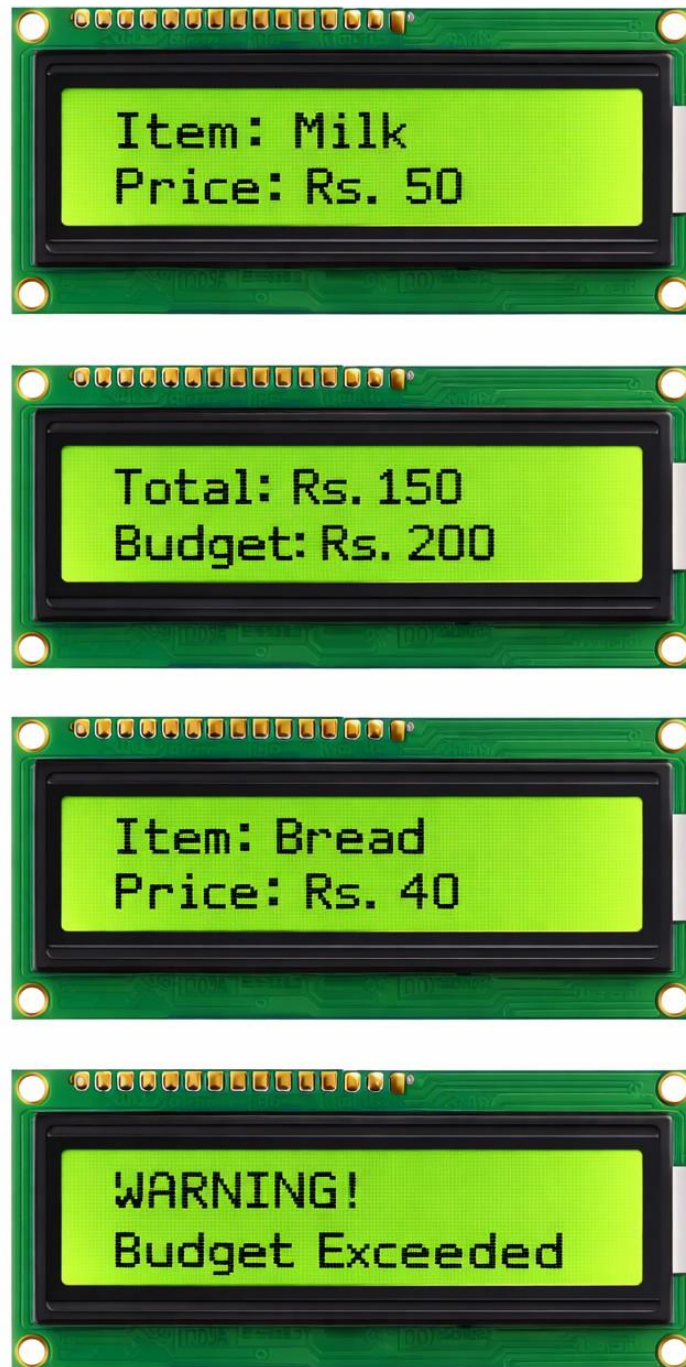
Fig. 4.2: Hardware Setup of Smart Trolley

5. RESULTS

The Smart Shopping Trolley system was successfully implemented and tested. The RFID reader accurately detected product tags and retrieved the correct item details from the database. The LCD displayed the product name and updated the total cost in real time. The keypad allowed smooth budget entry and control operations.

When the total exceeded the set budget, the buzzer alert and warning message worked effectively. The system also allowed the user to override the budget or cancel the item. Bluetooth communication successfully transmitted billing details to a mobile device. The servo motor responded correctly by opening the lid after each successful scan. Overall, the system performed efficiently and reduced manual billing effort.

5.1 LCD Output Display



Process Steps:

1. RFID Tag on product is brought near the RFID Reader.
2. Reader detects the tag and reads the UID.
3. UID is sent to ESP32 via SPI communication.

Fig. 5.1: LCD Output Display

6. CONCLUSION

The Smart Shopping Trolley system provides a modern and efficient solution for automated billing in supermarkets. It helps users track their expenses in real time and avoid overspending. By using RFID technology and ESP32, the system reduces billing time and improves accuracy.

The project demonstrates how embedded systems can enhance everyday shopping experiences. It is cost-effective, user-friendly, and reliable. In the future, the system can be improved by adding mobile applications, digital payment options, and a larger product database for real-time store integration.

7. REFERENCES

1. Arduino Documentation, "Arduino Programming Guide." Available: <https://www.arduino.cc>
2. Espressif Systems, "ESP32 Datasheet." Available: <https://www.espressif.com>
3. NXP Semiconductors, "MFRC522 RFID Reader Datasheet."
4. K. V. Shibu, Introduction to Embedded Systems, McGraw Hill Education, 2009.
5. LiquidCrystal I2C Library Documentation, Arduino Libraries.
6. Keypad Library Documentation, Arduino Playground.
7. SPI Communication Protocol Guide, Arduino Documentation.
8. BluetoothSerial Library for ESP32, Espressif Documentation.

Figures included in this project document

- Fig. 2.1 – Block Diagram of Smart Shopping Trolley System
- Fig. 2.2 – RFID Scanning Process
- Fig. 4.1 – Circuit Diagram of the System
- Fig. 4.2 – Hardware Setup of Smart Trolley
- Fig. 5.1 – LCD Output Display

8. SOURCE CODE

Arduino IDE source code used for the Smart Trolley System using ESP32 and RFID. This appendix contains the code supplied for the project.

```
#include <SPI.h>
#include <MFRC522.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
#include <ESP32Servo.h>
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

// RFID
#define SS_PIN 5
#define RST_PIN 22
MFRC522 mfrc522(SS_PIN, RST_PIN);

// LCD
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Servo
Servo myServo;
#define SERVO_PIN 4

// Budget
int budget = 0;
int total = 0;

// Keypad
const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};

byte rowPins[ROWS] = {32, 33, 25, 26};
byte colPins[COLS] = {27, 14, 12, 13};

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

// Product database
struct Product {
  String uid;
  String name;
  int price;
};

Product products[] = {
  {"12345678", "Milk", 50},
  {"87654321", "Bread", 30},
  {"11223344", "Rice", 80}
};

int numProducts = 3;

// Get UID as String
String getUID() {
  String content = "";
  for (byte i = 0; i < mfrc522.uid.size; i++) {
    content += String(mfrc522.uid.uidByte[i], HEX);
  }
  content.toUpperCase();
  return content;
}

// Find product
int findProduct(String uid) {
  for (int i = 0; i < numProducts; i++) {
    if (products[i].uid == uid) return i;
  }
  return -1;
}

// Get budget input
void setBudget() {
  lcd.clear();
  lcd.print("Set Budget:");

  String input = "";

  while (true) {
    char key = keypad.getKey();

    if (key) {
      if (key == '#') break;
      if (isdigit(key)) {
        input += key;
        lcd.setCursor(0,1);
        lcd.print(input);
      }
    }
  }
}
```

```

    budget = input.toInt();
    lcd.clear();
    lcd.print("Budget Set:");
    lcd.setCursor(0,1);
    lcd.print(budget);
    delay(2000);
}

// Servo open/close
void openLid() {
    myServo.write(90);
    delay(3000);
    myServo.write(0);
}

// Send via Bluetooth
void sendBill(String item, int price) {
    SerialBT.println("Item: " + item);
    SerialBT.println("Price: " + String(price));
    SerialBT.println("Total: " + String(total));
    SerialBT.println("-----");
}

void setup() {
    Serial.begin(115200);
    SerialBT.begin("SmartTrolley");

    SPI.begin();
    mfrc522.PCD_Init();

    lcd.init();
    lcd.backlight();

    myServo.attach(SERVO_PIN);
    myServo.write(0);

    lcd.print("Smart Trolley");
    delay(2000);

    setBudget();
}

void loop() {
    // RFID scan
    if (!mfrc522.PICC_IsNewCardPresent()) return;
    if (!mfrc522.PICC_ReadCardSerial()) return;

    String uid = getUID();
    int index = findProduct(uid);

    if (index != -1) {
        String name = products[index].name;
        int price = products[index].price;

        total += price;

        lcd.clear();
        lcd.print(name);
        lcd.setCursor(0,1);
        lcd.print("Total: " + String(total));

        sendBill(name, price);

        openLid();

        // Budget warning
        if (total > budget) {
            lcd.clear();
            lcd.print("Over Budget!");
            delay(2000);
        }
    } else {
        lcd.clear();
        lcd.print("Unknown Item");
    }

    delay(1500);
}

```